

Here is the production-ready code implementation for **Stage 4: Timeout, Retry Policy, Cost Logging, and Kill Switch**.

This iteration hardens the endpoint against upstream dependency failures, throttles, and budget spikes.

1. Updated Route Logic (`src/routes/your-thing.js`)

This implementation features exponential backoff with jitter, explicit error filtering (no retries on auth/client errors), standard execution timeouts, production execution logging, and an immediate infrastructure kill switch.

javascript

```
import express from 'express';
import fs from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';
import { z } from 'zod';
import { JobCardOutputSchema, stubResponse } from '../llm/schema.js';

const router = express.Router();
const __dirname = path.dirname(fileURLToPath(import.meta.url));
const PROMPT_VERSION = "job-classifier-v1";

// Schema for tracking metrics
const metricsPath = path.join(__dirname, '../logs/metrics.jsonl');
const quarantinePath = path.join(__dirname, '../logs/quarantine.jsonl');

// Ensure system logs directories are present
[metricsPath, quarantinePath].forEach(fp => {
  if (!fs.existsSync(path.dirname(fp))) {
    fs.mkdirSync(path.dirname(fp), { recursive: true });
  }
});

const RequestInputSchema = z.object({
  rawText: z.string({ required_error: "rawText is required" }).max(5000),
  userId: z.string({ required_error: "userId is required" })
});

// Helper: Read Prompt File
async function getSystemPrompt() {
```

```
const promptPath = path.join(__dirname, `../../prompts/${PROMPT_VERSION}.md`);
return fs.promises.readFile(promptPath, 'utf-8');
}

// Helper: Clean & Parse JSON
function cleanAndParseJSON(rawText) {
  let cleanText = rawText.trim();
  if (cleanText.includes("`")) {
    const match = cleanText.match(/``(?:json)?\s*([\s\S]*?)\s*``/);
    if (match) cleanText = match[1].trim();
  }
  return JSON.parse(cleanText);
}

// Helper: Write Performance and Cost Analytics
async function logMetrics({ model, promptTokens, completionTokens, durationMs, repaired }) {
  const metricLog = JSON.stringify({
    timestamp: new Date().toISOString(),
    promptVersion: PROMPT_VERSION,
    model,
    promptTokens,
    completionTokens,
    durationMs,
    repaired
  });
  await fs.promises.appendFile(metricsPath, metricLog + '\n', 'utf-8');
}

// Helper: Quarantine Failure Logs
async function quarantineFailure(inputData, rawModelOutput, errorDetails) {
  const logEntry = JSON.stringify({
    timestamp: new Date().toISOString(),
    promptVersion: PROMPT_VERSION,
    input: inputData,
    rawOutput: rawModelOutput,
    error: errorDetails
  });
  await fs.promises.appendFile(quarantinePath, logEntry + '\n', 'utf-8');
}

// Helper: Promise-based delay with Jitter
const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));

// Core Network Invocation wrapper handling explicit Timeouts, Error Filtering, and Backoff
```

```
async function callModelWithRetry(systemPrompt, userPayload, messagesHistory = [], maxRetr
const modelName = "google/gemini-2.5-flash";
const messages = [
  { role: "system", content: systemPrompt },
  { role: "user", content: JSON.stringify({ content: userPayload }) },
  ...messagesHistory
];

let attempt = 0;

while (attempt <= maxRetries) {
  const startTime = Date.now();
  const controller = new AbortController();
  const timeoutId = setTimeout(() => controller.abort(), 30000); // 30-second strict HTT

  try {
    const response = await fetch("https://openrouter.ai", {
      method: "POST",
      headers: {
        "Authorization": `Bearer ${process.env.OPENROUTER_API_KEY}`,
        "Content-Type": "application/json",
        "HTTP-Referer": "http://localhost:3000",
        "X-Title": "SaaS Hardened Classifier"
      },
      body: JSON.stringify({
        model: modelName,
        temperature: 0.1,
        messages: messages
      }),
      signal: controller.signal
    });

    clearTimeout(timeoutId);

    // Handle Success
    if (response.ok) {
      const data = await response.json();
      const durationMs = Date.now() - startTime;

      // Expose cost metrics accurately from the payload
      const usage = data.usage || { prompt_tokens: 0, completion_tokens: 0 };

      return {
        content: data.choices[0].message.content,
```

```

    metrics: {
      model: modelName,
      promptTokens: usage.prompt_tokens,
      completionTokens: usage.completion_tokens,
      durationMs
    }
  };
}

// Handle Client Errors that MUST NEVER be retried (Auth, Bad Data, Forbidden Action
if (response.status === 400 || response.status === 401 || response.status === 403) {
  const errText = await response.text();
  const error = new Error(`Fatal API Client Error (${response.status}): ${errText}`)
  error.status = response.status;
  throw error;
}

// Handle Server Overload (429 / 5xx) - Eligible for backoff retries
if (response.status === 429 || (response.status >= 500 && response.status <= 599)) {
  attempt++;
  if (attempt > maxRetries) {
    throw new Error(`Upstream system exhausted available processing attempts. Status
  }

// Honor Retry-After headers if delivered, otherwise drop back to custom exponenti
const retryAfterHeader = response.headers.get("retry-after");
let delayMs = Math.pow(2, attempt) * 1000 + (Math.random() * 300); // Exponential

if (retryAfterHeader) {
  const parsedSeconds = parseInt(retryAfterHeader, 10);
  if (!isNaN(parsedSeconds)) delayMs = parsedSeconds * 1000;
}

await sleep(delayMs);
continue;
}

throw new Error(`Uncategorized API response behavior encountered. Status: ${response

} catch (err) {
  clearTimeout(timeoutId);

// If it's a known non-retryable status error, throw it upward immediately
if (err.status) throw err;

```

```

    // Handle Abort / Timeout Exceptions
    if (err.name === 'AbortError') {
        attempt++;
        if (attempt > maxRetries) {
            const timeoutErr = new Error("Gateway connection target exceeded processing wind
            timeoutErr.status = 504;
            throw timeoutErr;
        }
        await sleep(Math.pow(2, attempt) * 1000 + (Math.random() * 300));
        continue;
    }

    // Generic network disconnect drop-outs
    attempt++;
    if (attempt > maxRetries) throw err;
    await sleep(Math.pow(2, attempt) * 1000 + (Math.random() * 300));
}
}
}

router.post('/your-thing', async (expressReq, expressRes) => {
    // 1. Immediate Input Validation Boundary
    let validatedInput;
    try {
        validatedInput = RequestInputSchema.parse(expressReq.body);
    } catch (err) {
        return expressRes.status(400).json({ error: "Validation Failed", invalidFields: err.er
    }

    // 2. Emergency Operational Kill Switch Boundary
    if (process.env.LLM_ENABLED === 'false') {
        return expressRes.status(503).json({
            error: "Service Temporarily Unavailable",
            message: "The AI processing module is disabled for maintenance. Please retry request
        });
    }

    // 3. Short Circuit Sandbox Configuration
    if (process.env.LLM_STUB === '1') {
        return expressRes.status(200).json(stubResponse);
    }

    let systemPrompt;

```

```

try {
  systemPrompt = await getSystemPrompt();
} catch (err) {
  return expressRes.status(500).json({ error: "Prompt specification file missing" });
}

let currentRawOutput = "";
let trackedError = "";
let runWasRepaired = false;

try {
  // Attempt 1: Normal execution cycle
  const executionOne = await callModelWithRetry(systemPrompt, validatedInput.rawText);
  currentRawOutput = executionOne.content;

  try {
    const parsedJson = cleanAndParseJSON(currentRawOutput);
    const schemaCheck = JobCardOutputSchema.safeParse(parsedJson);

    if (schemaCheck.success) {
      await logMetrics({ ...executionOne.metrics, repaired: false });
      return expressRes.status(200).json(schemaCheck.data);
    }
    trackedError = JSON.stringify(schemaCheck.error.errors);
  } catch (parseErr) {
    trackedError = `JSON Syntax Break: ${parseErr.message}`;
  }

  // Attempt 2: Bounded Repair Logic Action Cycle
  runWasRepaired = true;
  const repairHistory = [
    { role: "assistant", content: currentRawOutput },
    {
      role: "user",
      content: `Your previous answer was rejected for this reason: ${trackedError}. Retu
    }
  ];

  const executionTwo = await callModelWithRetry(systemPrompt, validatedInput.rawText, re
  currentRawOutput = executionTwo.content;

  const repairParsedJson = cleanAndParseJSON(currentRawOutput);
  const repairSchemaCheck = JobCardOutputSchema.safeParse(repairParsedJson);

```

```

if (repairSchemaCheck.success) {
  // Accumulate tokens used across both processing sessions for accurate cost logging
  await logMetrics({
    model: executionTwo.metrics.model,
    promptTokens: executionOne.metrics.promptTokens + executionTwo.metrics.promptToken
    completionTokens: executionOne.metrics.completionTokens + executionTwo.metrics.com
    durationMs: executionOne.metrics.durationMs + executionTwo.metrics.durationMs,
    repaired: true
  });
  return expressRes.status(200).json(repairSchemaCheck.data);
}

trackedError = JSON.stringify(repairSchemaCheck.error.errors);

} catch (globalErr) {
  if (globalErr.status === 401 || globalErr.status === 403) {
    return expressRes.status(globalErr.status).json({ error: "Upstream Authentication Fa
  }
  if (globalErr.status === 504) {
    return expressRes.status(504).json({ error: "Gateway Timeout", message: globalErr.me
  }
  trackedError = `Fatal processing pipeline breakdown: ${globalErr.message}`;
}

// Quarantine unresolvable failures cleanly
await quarantineFailure(validatedInput, currentRawOutput, trackedError);
return expressRes.status(422).json({
  error: "Unprocessable Entity",

```

Use code with caution.

message: "Pipeline components failed to guarantee data schema contracts."

});

});

export default router;

```
### 2. Documentation Architecture (`README.md`)
```

Append this configuration data block explicitly mapping implementation parameters to your

```
```markdown
```

```
Stage 4 Production Engineering Policy Specification
```

```
1. Underlying SDK Retry Override Policy
```

- We explicitly bypass internal SDK retry defaults by using a raw vanilla Node `fetch` loop
- This gives us clear line-of-sight control over our billing profiles by stopping hidden t

```
2. Retry Metrics Summary
```

- **Network Failures & Overloads (429, 5xx, Timeouts)**: Automatically retried up to 3 times
- **Client & Auth Restrictions (400, 401, 403)**: Dropped immediately. Failed fast without
- **Upstream Connection Timeouts**: Hardened to a 30-second window. Triggers a structured

```
3. Kill Switch Capabilities
```

- Setting the environment flag `LLM\_ENABLED=false` halts all downstream traffic before it

---

## Checkpoint Verification Scenarios

### Scenario 1: Kill Switch Enforcement (503 Service Unavailable Verification)

Set your context variables to include `LLM_ENABLED=false`, ensure `LLM_STUB` is clear, and launch your API instance.

#### Terminal Execution Command:

```
bash
```

```
LLM_ENABLED=false OPENROUTER_API_KEY=mock_key node src/index.js
```

Use code with caution.

```
bash
```

```
curl -i -X POST http://localhost:3000/your-thing \
-H "Content-Type: application/json" \
```



```
-d '{"rawText": "Testing emergency production cutoff actions.", "userId": "user_202"}'
```

Use code with caution.

### Expected HTTP Response Stream:

http

HTTP/1.1 503 Service Unavailable

Content-Type: application/json; charset=utf-8

```
{
 "error": "Service Temporarily Unavailable",
 "message": "The AI processing module is disabled for maintenance. Please retry requests"
}
```

Use code with caution.

*(Reviewing logs confirms absolutely zero model queries were transmitted.)*

### Scenario 2: Broken Authentication Handling (Fast-Fail 401 Verification)

Restore `LLM_ENABLED=true` but purposely contaminate your authentication keys inside your active profile.

### Terminal Execution Command:

bash

```
LLM_ENABLED=true OPENROUTER_API_KEY=sk-or-v1-EXPIRED-BAD-KEY node src/index.js
```

Use code with caution.

bash

```
curl -i -X POST http://localhost:3000/your-thing \
 -H "Content-Type: application/json" \
 -d '{"rawText": "Verify fast authentication rejection boundaries.", "userId": "user_202"
```

Use code with caution.

### Expected HTTP Response Stream:

http

HTTP/1.1 401 Unauthorized

Content-Type: application/json; charset=utf-8

```
{
 "error": "Upstream Authentication Failure",
 "message": "Fatal API Client Error (401): Invalid API Key configuration or profile valid
}
```

Use code with caution.

*(Reviewing execution console timelines indicates the request completes instantly—proving zero retries were attempted on the invalid auth key).*

---

## Checkpoint Complete 🏁

- Strict 30s connection timeout stops leaking orphaned sockets.
- Token metrics collection pipeline is live in `logs/metrics.jsonl`.
- Backoff logic handles system overloads gracefully without compounding failures.

I have completed the requested changes. Would you like to proceed with either of these options?

↪ Simulate a **synthetic 429 Throttle Response** to watch the retry jitter in action

↪ Set up **Prometheus-style metric exporters** for the cost tracking schema